

some-code-errors

Documentation: Project Debugging and Problem Solving

This document details the series of challenges encountered and the solutions implemented to build a robust, automated pipeline for lunar elemental mapping.

1. Python Environment and Scripting Errors

Problem: Incompatible Libraries

- **Symptom:** The script would crash immediately upon trying to import `scipy`, with a `ValueError: numpy.dtype size changed, may indicate binary incompatibility.`
- **Diagnosis:** This indicated a conflict between the system-wide installation of Python libraries. An old version of `SciPy` installed by the OS was incompatible with a newer version of `NumPy` installed by the user.
- **Solution:** A dedicated **Python virtual environment** (`venv`) was created for the project. This isolated the project's dependencies from the system. Fresh, compatible versions of all required libraries (`numpy`, `scipy`, `pandas`, `astropy`, etc.) were installed into this clean environment using `pip`.

Problem: `ImportError` and `NameError`

- **Symptom:** The main batch script would fail with `ImportError: cannot import name ...` or `NameError: name '...' is not defined`.
- **Diagnosis:** During the iterative debugging process, helper scripts were updated with new function names or logic. However, the main `run_analysis_batch.py` script (or another helper script) was not always updated at the same time, causing it to call old, non-existent functions or reference variables that were no longer in scope.
- **Solution:** The final solution was to use the foolproof `cat > ... << EOF` command to directly overwrite each of the project's Python scripts with their final,

definitive, and mutually compatible versions, ensuring the entire codebase was synchronized.

2. XSPEC Scripting and Automation Failures

Automating the `XSPEC` software proved to be the most challenging part of the project.

Problem: Silent Failures

- **Symptom:** The `run_analysis_batch.py` script would execute the `XSPEC` step, which would start and immediately quit with no output or error messages.
- **Diagnosis:** This was caused by two issues:
 1. Using the wrong command (`xspec - script.cmd`), which is for an advanced scripting language (TCL), not the simple command list we were using.
 2. The script file having incorrect line endings (Windows `CRLF` instead of Linux `LF`) due to copy-pasting, which `XSPEC` could not parse.
- **Solution:**
 1. The command was corrected to use input redirection (`xspec < script.cmd`), which correctly feeds the commands to the program.
 2. The `dos2unix` utility was used to fix the file's line endings.

Problem: Incomplete Input Files (`.pha` header errors)

- **Symptom:** `XSPEC` would fail to load the `.pha` file with errors like `data file missing` `required entries: Keyword BACKFILE is missing...` , followed by `AREASCAL` , and `BACKSCAL` .
- **Diagnosis:** The `prepare_xsm_for_xspec.py` script was generating `.pha` files with an incomplete FITS header. `XSPEC` requires a full set of OGIP-standard keywords to properly understand a spectrum file.
- **Solution:** The `prepare_xsm_for_xspec.py` script was updated to model the header of an official ISRO CLASS data file. All required keywords (`BACKFILE` , `RESPFILE` , `AREASCAL` , `BACKSCAL` , `CORRSCAL` , `POISSERR` , etc.) were added, making the generated `.pha` files fully compliant.

Problem: Calibration File Not Found

- **Symptom:** `XSPEC` would fail with `Error: cannot read response file...`.
- **Diagnosis:** The `.pha` file's header contained only the filename (e.g., `ch2xsm...rsp`) of the calibration files, not the full path. When `XSPEC` ran, its working directory was different from the calibration directory, so it couldn't find the files. A bug in the Python script using `.name` instead of the full path object was identified.
- **Solution:** The `prepare_xsm_for_xspec.py` script was corrected to write the **full, absolute path** to the `BACKFILE` and `RESPFILE` in the `.pha` header, making the files self-contained.

Problem: Interactive Hangs and Bad Fits

- **Symptom:** The `XSPEC` script would get stuck, or produce `invalid parameter input` errors, or run to completion but produce a meaningless fit with "frozen" parameters.
- **Diagnosis:** The `model apec` command in `XSPEC` is interactive by default; it pauses to ask for parameter values. The script was not providing this input, causing it to fail. Furthermore, even when it ran, the default model parameters were "frozen," meaning the `fit` command couldn't actually change them to find a good solution.
- **Solution:** The `run_analysis_batch.py` script was updated to generate a fully **non-interactive** `XSPEC` script. It explicitly provided initial values and step sizes for the model parameters (`kt` and `norm`), which "thaws" them and allows the `fit` command to work correctly.

3. `pyxspec` Implementation Failures

To overcome the fragility of generating external scripts, the project was migrated to use `pyxspec`, the Python interface for `XSPEC`.

Problem: `AttributeError` and Silent Failures

- **Symptom:** The `pyxspec` script would crash with `AttributeError` (e.g., on `.clear()` or `.modelFlux`), or it would run to completion but produce an empty output file, leading to a `No valid data points` error in the next step.

- **Diagnosis:** This was caused by several bugs in the implementation:
 1. Using incorrect method names to control `pyxspec` (e.g., `Xset.clear()` instead of `AllData.clear()`).
 2. Trying to extract the final data from the wrong `pyxspec` object.
 3. Relying on `XSPEC` 's plotting system to get data, which proved unreliable in a non-interactive script.
 4. Most importantly, the scientific fit was still failing silently, producing a model with zero flux, which resulted in empty data arrays being returned.
 - **Solution:** The `run_xspec_analysis_with_pyxspec` function was completely rewritten.
 1. The logic was simplified to use a single, robust `apec` model.
 2. A crucial **diagnostic check** was added *after* the `xspec.Fit.perform()` command to verify that the fit resulted in a physically meaningful, non-zero normalization.
 3. The method for extracting the final data was changed to a more direct method, pulling the energy and flux arrays from the core `Spectrum` object (`s.energies` , `s.modelFlux`) and later from the `Plot` object (`Plot.x(1)` , `Plot.model(1)`), bypassing the most fragile parts of the library.
-

4. Final Scientific and Plotting Errors

Problem: Incorrect Continuum Subtraction

- **Symptom:** The diagnostic plot for the continuum subtraction showed a flat model that did not match the data.
- **Diagnosis:** The `scipy.optimize.minimize` function was failing to find the correct scaling factor for the continuum model because its initial guess was too far from the true value.
- **Solution:** The script was made smarter by adding logic to calculate a much better initial guess based on the data itself, ensuring the fitter would converge correctly.

Problem: Unidentified Image Error

- **Symptom:** The final mapping script crashed with `PIL.UnidentifiedImageError`.
- **Diagnosis:** The script was pointed to a `.IMG` file, which is a raw scientific data format, not a standard image format like `.PNG` or `.JPG`.
- **Solution:** The user was directed to download the correct `.PNG` version of the lunar base map and update the path in the script.